

HTB ACADEMY

Working with IDS/IPS

HTB Academy · Module 226 · Detection Engineering Track · Walkthrough

A complete illustrated walkthrough of Suricata, Snort 3, and Zeek — covering rule development for real C2 frameworks and malware, JA3 TLS fingerprinting, and network security monitoring. Lab answers are partially masked as a hint.

Platform	HackTheBox Academy
Module	Working with IDS/IPS (#226)
Completion	100% — all 10 sections
Analyst	Chayan Panchal · @tron874
Core Skills	Suricata, Snort 3, Zeek, rule development, JA3 fingerprinting, NSM

1 IDS vs. IPS — Core Concepts

Feature	IDS	IPS
Mode	Passive — alerts only	Active — blocks inline
Network position	Out of band, behind firewall	Inline — all traffic passes through
Traffic impact	None	Can add latency
On detection	Generates alert	Drops packets, resets connections
False positive risk	Low impact	High — can block legitimate traffic

Detection methods

Method	How it works
Signature-based	Matches known patterns — high precision, misses zero-days
Anomaly-based	Detects deviation from baseline — catches unknowns, more false positives
Stateful protocol analysis	Tracks protocol state — catches misuse and protocol violations

Tools covered

Tool	Version	Role
Suricata	6.0.13	IDS/IPS/NSM — primary alert engine, rule-based detection
Snort 3	3.1.64	IDS/IPS — Lua-based config, complementary rule engine
Zeek	Latest	NSM — structured log generation for all protocols

2 Suricata — Commands, Output & Rule Anatomy



Suricata — key commands

```
suricata -r pcaps/suspicious.pcap          # offline PCAP analysis
sudo suricata -i ens160                    # live capture, AF_PACKET
sudo suricata -T -c /etc/suricata/suricata.yaml # validate config
sudo suricata-update && suricata-update list-sources
sudo kill -usr2 $(pidof suricata)          # reload rules live
```

Output file	Contents
eve.json	All events — alerts, DNS, HTTP, TLS, flows, file metadata
fast.log	Alerts only — timestamp, SID, message, src/dst
filestore/	Files extracted from traffic, named by SHA256 hash



Filtering eve.json with jq

```
cat eve.json | jq -c 'select(.event_type == "alert")'
cat eve.json | jq -c 'select(.event_type == "http")' | jq .flow_id
```

Rule anatomy — action protocol src -> dst (options;)

Option	Purpose	Example
msg	Alert message shown to analyst	msg:"ET MALWARE Dridex C2";
content	Byte string to match in payload	content:"User-Agent: Go-http-client";
http_uri / http_method / http_cookie	HTTP sticky buffers — Suricata syntax	http_uri; content:"/serial.php";
ja3.hash	Match a TLS JA3 fingerprint	ja3.hash; content:"473cd7cb...";
detection_filter	Threshold before an alert fires	detection_filter:track by_src,count 4,seconds 10;

3 Detection Rules Written

```
PowerShell Empire C2 — Suricata (HTTP)

alert http $HOME_NET any -> $EXTERNAL_NET any
(msg:"ET MALWARE Possible PowerShell Empire Activity Outbound";
 flow:established,to_server;
 content:"GET"; http_method;
 pcre:"/^(?:login\/process|admin\/get|news)\\.php$/RU";
 content:"session="; http_cookie;
 sid:2027512; rev:1;)
```

```
Covenant C2 — anomaly detection by beacon size

alert tcp $HOME_NET any -> any any
(msg:"detected by size and counter";
 dsize:312;
 detection_filter: track by_src, count 3, seconds 10;
 priority:1; sid:3000001;)
```

```
Sliver C2 — JA3 TLS fingerprint match

alert tls any any -> any any
(msg:"Sliver C2 SSL";
 ja3.hash; content:"473cd7cb9faa642487833865d516e578";
 sid:1002; rev:1;)
```

```
Patchwork APT — HTTP C2 (Snort, sticky buffers)

alert http $HOME_NET any -> $EXTERNAL_NET any
(msg:"OISF TROJAN Targeted AutoIt FileStealer CnC Beacon";
 flow:established,to_server;
 http_method; content:"POST";
 http_uri; content:".php?profile=";
 http_client_body; content:"ddager=", depth 7;
 http_header; content:!"Referer|3a|";
 sid:10000006; rev:1;)
```

CAUTION HTTP sticky buffers are tool-specific — `http_header`; is correct for Snort rules, `http_user_agent`; is Suricata-specific. Using the wrong keyword in the wrong engine means the rule silently never fires.

4 Snort 3 & Zeek



Snort 3 — key commands

```
snort -c snort.lua --daq-dir /usr/local/lib/daq # validate config
sudo snort -c snort.lua -r file.pcap -A cmg # run on PCAP, verbose
sudo snort -c snort.lua -i ens160 # live capture
```

Zeek log	Contents	Key fields
conn.log	All connections	id.orig_h, id.resp_h, proto, duration
dns.log	DNS queries/responses	query, answers, rcode, TTL
ssl.log	TLS session metadata	server_name, cipher, version, ja3
x509.log	X.509 certificate details	certificate.subject, certificate.issuer
dce_rpc.log	DCE/RPC calls (WMI, SMB)	endpoint, operation, id.resp_p



Zeek — CLI reference

```
zeek -r pcaps/suspicious.pcap # run on PCAP
cat http.log | zeek-cut host uri user_agent # extract columns
ja3 -a --json pcaps/sliverenc.pcap # JA3 from PCAP
zgrep "malicious-domain.com" 2023-07-06/dns.log.gz # search compressed logs
```

NOTE Zeek's dce_rpc.log captures WMI and lateral movement traffic — always worth checking first during an IR involving Windows lateral movement.

5 Lab Answers & Key Takeaways



All lab answers (masked)

```
Suricata Fundamentals Q1 – flow_id from HTTP events      -> 1.....0.....3
Suricata Fundamentals Q2 – PHP page in http.log          -> a....p
Suricata Rule Dev 1   Q1 – min offset, EternalBlue rule  -> 4
Suricata Rule Dev 2   Q1 – JA3 hash, Trickbot TLS rule   -> 7.....f.....a
Snort Fundamentals    Q1 – SID trigger count, wannamine   -> 234
Snort Rule Dev        Q1 – keyword before content, log4shell -> h.....e....;
Zeek Fundamentals      Q2 – port in dce_rpc.log for WMI    -> 2*11
Skills Assmt Suricata Q1 – content payload for WMI rule   -> C*a.e
Skills Assmt Snort    Q1 – RC4-HMAC Kerberos etype value  -> 17
Skills Assmt Zeek     Q1 – x509.log field for CN          -> c.....t.....t
```

Takeaway	Why it matters
IDS alerts, IPS blocks	IPS must sit inline; IDS is out-of-band
Use all three together	Suricata, Snort, and Zeek serve different, complementary purposes
JA3 fingerprints encrypted C2	Hash the TLS ClientHello parameters — no decryption needed
detection_filter beats single-packet rules	Turns one match into a beaconing-pattern detection
RC4-HMAC (etype 17) in Kerberos = red flag	Modern Windows uses AES256 (etype 18) — RC4 usage suggests Overpass-the-Hash

6 Quick Reference — Commands



Suricata / Snort / Zeek — quick command index

```
suricata -r file.pcap
sudo suricata-update
snort -c snort.lua -r file.pcap -A cmg      # Snort on PCAP
zeek -r file.pcap
ja3 -a --json file.pcap

sudo suricata -i ens160
sudo kill -usr2 $(pidof suricata)
cat FILE.log | zeek-cut field1 field2
cat eve.json | jq -c 'select(.event_type=="alert")'
```